Can a small local agent redo a cloud agent's work?

93 evaluator-verified tasks from an agent-built health-data repository, with an open harness and a Gemma 4 E4B baseline

John Bowyer, Health Transformation Lab

Gemma 4 Developer Agent Competition, Paper Track [14].
Topic: Tasks & Benchmarks (resource paper). November 2026.

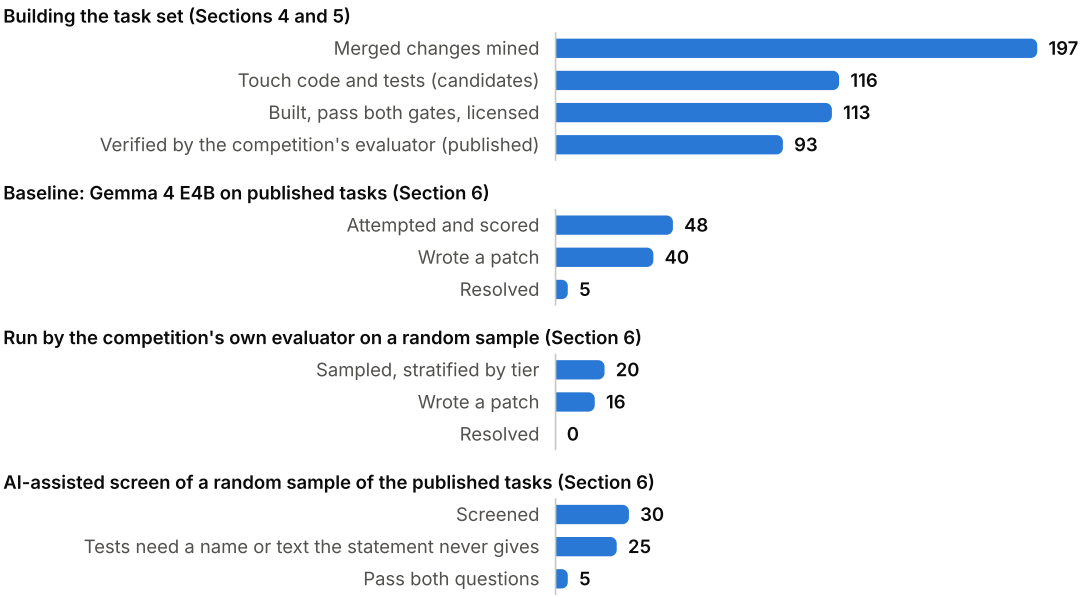
A personal learning project, not affiliated with or endorsed by any employer, institution or vendor. Every patient record in the source repository is synthetic.

ABSTRACT

Coding-agent benchmarks are mined from mature web libraries maintained by people. I release a task set mined from a different source: a health-data repository built by cloud coding-agent sessions under written rules that CI enforces. It has 93 tasks in the competition's format, with snapshots, code graphs and embeddings; the competition's own evaluator resolves every reference patch and no empty patch. **A verified task is not yet a fair one:** in 25 of 30 screened tasks, the tests need a name or text the statement never gives. **A small local model resolves few of them:** Gemma 4 E4B resolved 5 of 48 on two free T4 GPUs under tight limits, 2 of the 23 cheapest of those with eight times the output budget and thinking on, and none of a random 20 run end to end by the competition's evaluator on its hardware. **It leaves the code graph almost unused:** across three setups, 20 of 473 tool calls were to a graph tool. **The graph seldom leads to the fix:** names and call edges connect a task's statement to the code its fix edits in 22 of 93 tasks.

Data and code. Tasks (CC BY 4.0): kaggle.com/datasets/jcblearning/health-transformation-lab-coding-agent-tasks.
Quickstart notebook: kaggle.com/code/jcblearning/coding-agent-tasks-quickstart.
Harness, builders and run tables (Apache 2.0): github.com/healthtransformationlab/health-transformation-lab-nlp.
Agent configuration and each number's source (Apache 2.0): github.com/healthtransformationlab/gemma-developer-agent.

Figure 1. Task counts at each step, on one scale.



1. Introduction

The best coding agents today run on large models in the cloud. This competition [14] asks whether a model that fits on one accelerator can do the same work. The tasks available come from a narrow place: SWE-bench draws on twelve popular Python libraries [1], and the competition's development set on four.

Two kinds of repository are scarce. In a **data repository** the costly defect is silent: a pipeline finishes, a table is written, and a number in it is wrong. In the repository studied here (synthetic records throughout), a masking defect left about a quarter of charted teeth without measures while every test passed. An **agent-built repository**, written by agents under a person's direction, can leave an unusually complete record: an issue, a pull request and a review for each change. A large cloud agent did this work. How much of it can a small local agent redo from the same problem statements?

I study one repository that is both. `health-transformation-lab-nlp`, part of my open health-data benchmarks, exists to surface silent defects, here in how well gum disease can be found in synthetic dental records. I typed no code: every change was made by coding-agent sessions I directed in chat and merged after an automated review.

Contributions:

1. **A task set**: 93 tasks in the competition's format with code graphs and embeddings, each verified with the competition's own evaluator and labelled with its statement's source, public under CC BY 4.0 (Section 4).
2. **An open harness** that agrees with the competition's evaluator on every saved agent patch and keeps answers from the model (Section 5).
3. **A baseline under three settings** for Gemma 4 E4B, with a failure breakdown, costs and a paired test of the code-graph tools (Section 6).
4. **Three checks on the recipe**: whether statements say what their tests check (Section 6), whether the graph leads from statement to fix (Section 4.3), and a second repository (Section 4.5).

2. Related work

Task sets from repository history. SWE-bench pairs GitHub issues with the pull requests that closed them and scores a patch by the repository's tests [1]; SWE-bench Verified adds human screening of task quality [2]. SWE-Gym [5], SWE-smith [6] and SWE-rebench [7] scale task collection for training and for contamination-free evaluation. All draw on human-maintained libraries; this set applies the recipe to an agent-built data repository and labels each statement's source.

Agents and graphs. SWE-agent showed that the interface matters as much as the model [8]; the competition fixes it. RepoGraph [3] and LocAgent [4] lead an agent to the code to edit by finding an issue's terms in a code graph and walking its edges; Section 4.3 counts how often such a walk exists here.

Data work. DS-1000 tests data-science code generation [9]; Spider 2.0 [10], ELT-Bench [11] and data-eng-bench [13] score an artefact agents produce: SQL, a pipeline, dbt models. MLOps-Bench [12] is closest: 212 fail-to-pass tasks from 31 public ML repositories.

3. The source repository

The repository holds a Python package and tests, an R twin kept in numerical parity, SQL and Databricks pipelines, and documentation, written October 25 to November 8, 2026.

Who wrote it. Of 960 commits on `main`, 775 are marked as agent-authored; the rest are GitHub's merge commits for pull requests (175), a bot's (5), mine alone (4) and the initial commit.

Rules the agents work under. A checked-in `AGENTS.md` requires an issue per user story, a branch and pull request tied to it, and a test before the code; CI rejects a pull request without an issue link. Every pull request has a review committed as a file (240 of them).

4. Building the task set

4.1 Mining

The mining script reads every merged pull request and every first-parent commit on `main` as of October 29, 2026: 197 changes. A change is a **candidate** when its diff touches both non-test code and test files, which is the competition's own mining rule.

Step	Tasks
Merged changes	197
Touch code and tests (candidates)	116
Built (one withdrawn: its patch names an organisation the repository's rules forbid)	115
Pass both gates	114
One withheld (its patch adds third-party files with no stated licence)	113
Published: the competition's evaluator resolves the reference patch (Section 5)	93

4.2 Fidelity tiers

Each task records where its `problem_statement` came from.

Tier	Statement source	Candidates (published)
A	the issue body	49 (39)
B	the pull request's title and body (no issue existed)	45 (34)
C	the commit message (no pull request existed)	22 (20)

Tiers B and C exist because the issue rule was adopted partway through the history; for those 67 candidates a **backfilled issue record** quotes the original text verbatim.

4.3 Snapshots, graphs, embeddings

A snapshot is the tree at `base_commit` as a single commit, so no later history comes along. The code graph is an AST walk over the package into NetworkX node-link JSON with the competition's keys (median 488 nodes and 397 `calls` edges). Embeddings come from EmbeddingGemma, cut to 256 dimensions and renormalised. A format check against the competition's own files matched the snapshot layout, graph keys and embedding shape; the competition does not name its embedding model.

The graph reaches only part of what a data repository changes. In 31 of the 93 tasks the reference patch touches no module the graph covers (new modules, scripts, notebooks, configuration or documents), and in 11 more it changes no existing function or class. In the other 51, the statement names an edited function or class (by its identifier) in 20, call edges lead there from one it names in 2 more, and in 29 neither does, which leaves embedding search, untested here.

4.4 Gates

A task is kept only if its tests fail at `base_commit` with the test patch alone, and pass once the reference patch is applied with the files the grader strips removed. All 115 fail without the fix; 114 pass with it.

4.5 A second repository

Applied unchanged to Open Navigator, a public repository run by a nonprofit I lead, with several human contributors and no issue-tracing rule, the mining rule finds 133 candidates in 1,320 first-parent commits; none links an issue, and only 22 can be tied by commit subject to one of its 169 merged pull requests. Candidates are everywhere; statements depend on what the history recorded. The 133 are not gated or released.

5. Harness and verification

My harness. I re-implemented the harness the competition's guide documents before I found the competition's evaluator, which ships in a separate Kaggle dataset. Two scripted models bound it: replaying each reference patch from the repository's history resolves 111 of 113 tasks (the other two need Postgres and R), and submitting nothing resolves none.

The competition's evaluator. I then ran the competition's own verification (`swegemma` 0.2.11, subprocess sandbox, unmodified) on the 113 tasks first published and on the corrected release.

Snapshots	Reference patch	Empty patch	Gemma's saved patches
First release, one file added per snapshot	93 of 113 resolved	0 of 113	same verdict as my harness on 25 of 25
Corrected release, as published	93 of 93 resolved	0 of 93	same verdict as my harness on 23 of 23

As published, the first release could not be scored (all three tasks tried fail at import): the evaluator puts the repository root on the import path, and this package sits in `python/`. The corrected release adds a short `conftest.py` at each snapshot's root and keeps only the 93.

Of the 20 left out, twelve have tests that read third-party files the published snapshots withhold for licence reasons (my control ran from the repository's history and missed this), six start a script that imports a module beside it, which the evaluator's `PYTHONSAFEPATH=1` forbids and my harness allows, and two need Postgres or R. Elsewhere the two verifiers agree on every reference patch.

Keeping answers from the model. This repository's history contains every fix, and later snapshots contain earlier tasks' fixes. Commands and verification run in Linux namespaces with no network and an allow-listed environment. Kaggle GPUs do not allow namespaces, and a notebook can read every attached file, so each task runs alone in a private, offline notebook beside a private dataset holding that task only; the patch is verified locally.

Known differences. Namespaces instead of a container; no ADK context compaction or prefix caching; subagents and skills are not run; `PYTHONSAFEPATH` is not set. No number here is a competition score.

6. Baseline

Setup. `gemma-4-e4b-it` in fp16 with transformers on two T4 GPUs, internet off, thinking off, at most 2,048 new tokens per turn, a 24,576-token context, 40 tool calls, 60 turns and 30 minutes per task. Tasks were run cheapest first under a GPU-hour cap.

Results. Ten of the 58 tasks attempted are among the twenty left out (Section 5), so they are set aside. On the other 48 the model wrote a patch for 40 and resolved 5 (10.4%, 95% Wilson CI 4.5–22.2). By statement source it resolved 4 of 24 from issues, 1 of 19 from pull requests and 0 of 5 from commit messages; the intervals overlap.

Do the graph tools help? A harness fault served the first tasks without their code graph. After fixing it I reran 28 of them with the same settings; the 23 published give a paired comparison.

23 tasks, each run twice	Non-empty patch	Resolved (95% CI)
Graph tools unavailable	18	2 (8.7%, 2.4–26.8)
Graph tools available	20	2 (8.7%, 2.4–26.8)

One task was resolved only without the tools, one only with them, one both times. With the tools available the model called one in 4 of the 23 runs, 6 calls among 145: it reads and edits files and rarely searches.

How it fails. Of the 43 unresolved tasks, 35 submitted a patch that fails the target tests and 8 submitted nothing. Of those eight, four have a turn that hit the 2,048-token limit, three submit without an edit, and one calls the edit tool with an argument it does not take. The five resolved tasks are small: a CI rule, a warning message, a label parser and two settings.

A larger budget. To see whether the tight limits decide the result, I reran the 23 cheapest of the 48 on the same harness with 16,384 output tokens per turn, thinking on, 100 tool calls and 20 minutes (one L4 GPU on Colab, a 20,000-token context). The model resolved 2 of 23, against 2 under the tight limits, and wrote a patch for 20 against 19. One of the 23 runs had a turn above 2,048 output tokens (the longest, 3,335), and one reached the context limit. Of its 139 tool calls, 5 were to a graph tool. Colab notebooks have network access, which Kaggle's did not; no command in the transcripts used it.

Task fairness. An AI reviewer screened 30 of the 93 on SWE-bench Verified's two questions [2]; I checked its findings in summary and agreed with all 30. Five pass. In 25 the tests need a name or text the statement never gives. Of 16 the model attempted, it resolved 1 of the 3 that pass and none of the 13 that fail, so the baseline may measure the tasks more than the model.

Under the competition's evaluator. I also ran the agent end to end with the official harness: `swegemma`'s agent loop, vLLM from the competition's wheelhouse, four L4 GPUs, and my competition settings (16,384 output tokens, a 4,096-token thinking budget, 40 tool calls, 5 minutes). The sample is 20 of the 93, seeded and stratified by tier, not by cost. Each task ran alone, without its patch or tests. The agent resolved 0 of 20 (95% CI 0–16.1): 16 patches fail and 4 runs submit nothing. No run reached a budget. In 189 tool calls the model called a graph tool 9 times, the analyzer sub-agent 3 times and the skill never. Thirteen of the 20 are also in the screen above; 11 of them fail it. The run also exposed a defect my harness had hidden: my prompt required a hints variable, which ends the agent on the half of tasks with none (fixed before those ran).

Cost. The 58 first attempts and 28 reruns used 30.2 T4 GPU-hours, about $21 at an estimated $0.69 per hour, or 6.0 GPU-hours for each task resolved. The evaluator's 21 runs used 12.8 L4 GPU-hours and the larger-budget rerun 2.1.

7. Limitations

- **One repository, one author's rules.** The second repository tests only the mining rule.

- **Agent-written statements.** The issue text was largely written by a cloud agent at my direction, which may make it easier or harder to follow than a person's.
- **No leaderboard score.** My harness ran the baseline and the larger-budget rerun. The competition's evaluator ran the agent on 20 of my tasks only, with my settings (5 minutes and 40 tool calls against the default 60 and 100).
- **Small.** 48 of 93 tasks, cheapest first, and a random 20 under the evaluator; one sampled run per condition at temperature 0.2. One task flipping moves a paired result.
- **A screen checked only in summary.** I saw the AI's verdict first and did not read the tests; 63 tasks are unscreened.
- **The tasks are not silent.** Each comes with a failing test. The set measures repair of the code that looks for silent defects, not noticing one unprompted.

8. Conclusion and release

A cloud agent built this repository in fifteen days. From the same problem statements, a model that runs on free GPUs resolved 5 of 48 tasks under tight limits, all of them small; 2 of 23 with a larger budget; and none of a random 20 under the competition's own evaluator. Only 20 of 473 tool calls were to a graph tool, and names and call edges lead from statement to fix in 22 of 93 tasks. Whether graphs help a small model on a data repository cannot be told until the model asks them and they cover more than its package.

Using the set. The dataset has the competition's layout, so the competition's evaluator runs on it as published. In the repository, `kaggle/official_verify.py` replays the two controls and `63_graph_coverage.py` the graph counts; the quickstart notebook loads a task with its snapshot, graph and embeddings.

The recipe should transfer to any repository that traces changes to issues: mine changes that touch code and tests, record each statement's source, gate, run reference-patch and empty-patch controls on the published files with the official evaluator, and screen statements against tests before trusting any model's score.

References

1. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024.
2. Chowdhury et al. Introducing SWE-bench Verified. OpenAI, 2024.
3. Ouyang et al. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. ICLR 2025.
4. Chen et al. LocAgent: Graph-Guided LLM Agents for Code Localization. ACL 2025.
5. Pan et al. Training Software Engineering Agents and Verifiers with SWE-Gym. ICML 2025.
6. Yang et al. SWE-smith: Scaling Data for Software Engineering Agents. NeurIPS 2025.
7. Badertdinov et al. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. NeurIPS 2025.
8. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024.
9. Lai et al. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. ICML 2023.
10. Lei et al. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. ICLR 2025.
11. Jin, Zhu, Kang. ELT-Bench: An End-to-End Benchmark for Evaluating AI Agents on ELT Pipelines. PVLDB, 2026.
12. Tran et al. MLOps-Bench: Benchmarking AI Agents on Production ML Engineering in Real Repositories. arXiv:2610.23506, 2026.
13. Snowflake Inc. data-eng-bench: a data-engineering benchmark for coding agents. Version 1.0.0, github.com/Snowflake-Labs/data-eng-bench, 2026.
14. Markowitz, Perozzi, Rózemberczki, Cameron, Hemmati, Li, Galkin, Farhadi, Holbrook, Oldacre. Google - The Gemma 4 Developer Agent Competition. Kaggle, 2026. https://www.kaggle.com/competitions/gemma-4-developer-agent